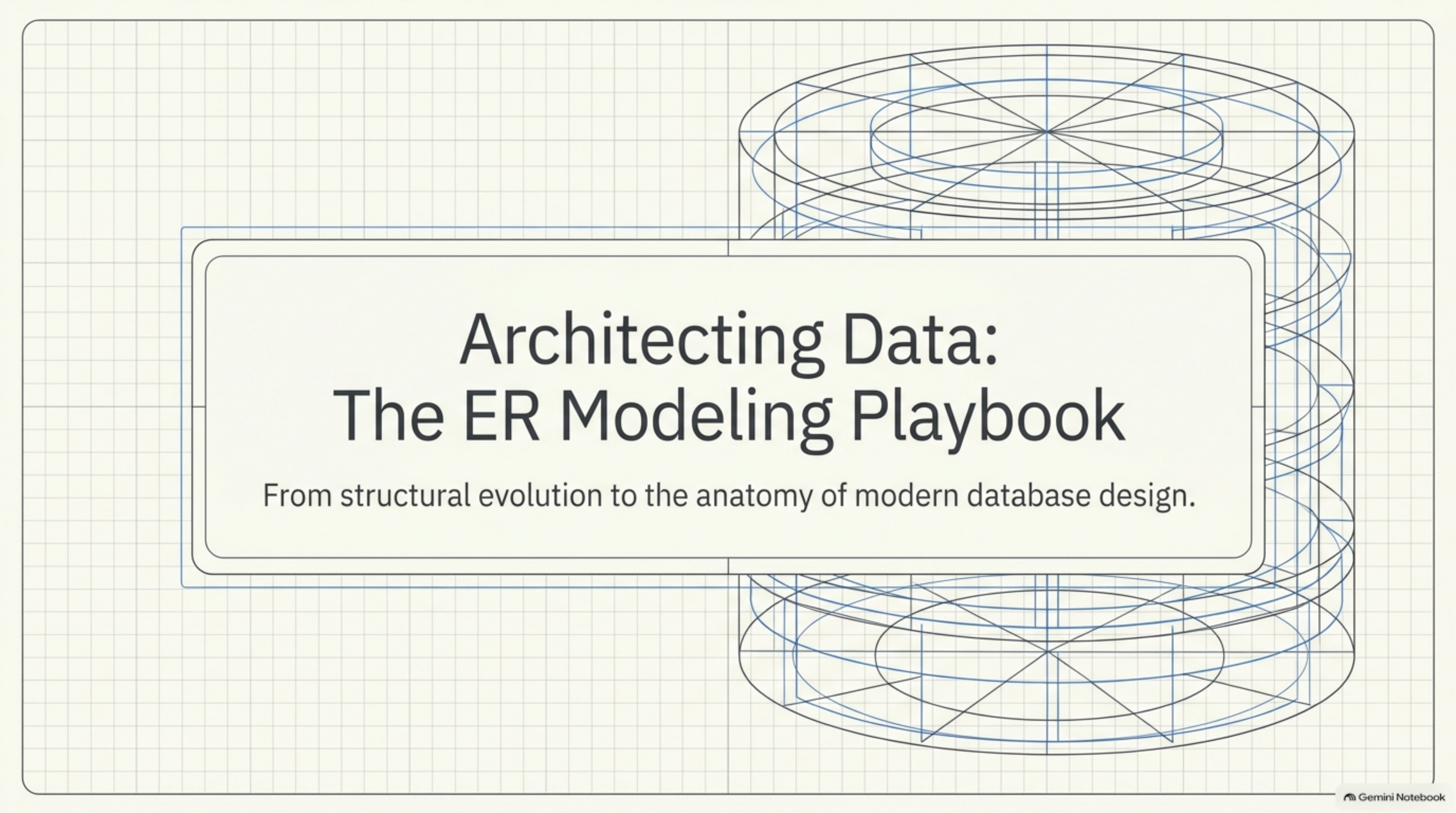




Architecting Data: The ER Modeling Playbook

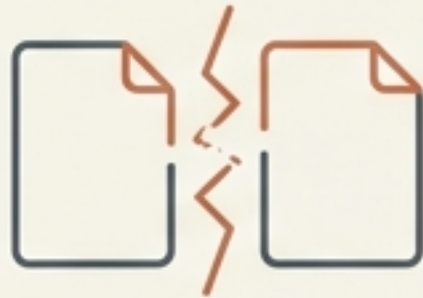
From structural evolution to the anatomy of modern database design.

The File-Based Dilemma (Ancient–1960s)

Early applications managed their own files independently, leading to systemic failures as data volume grew.

Isolated Data

Incompatible formats meant applications could not work together.



Redundancy

The exact same facts were duplicated across multiple disconnected locations.



Inaccessibility

Retrieval required writing custom programs for every individual file.

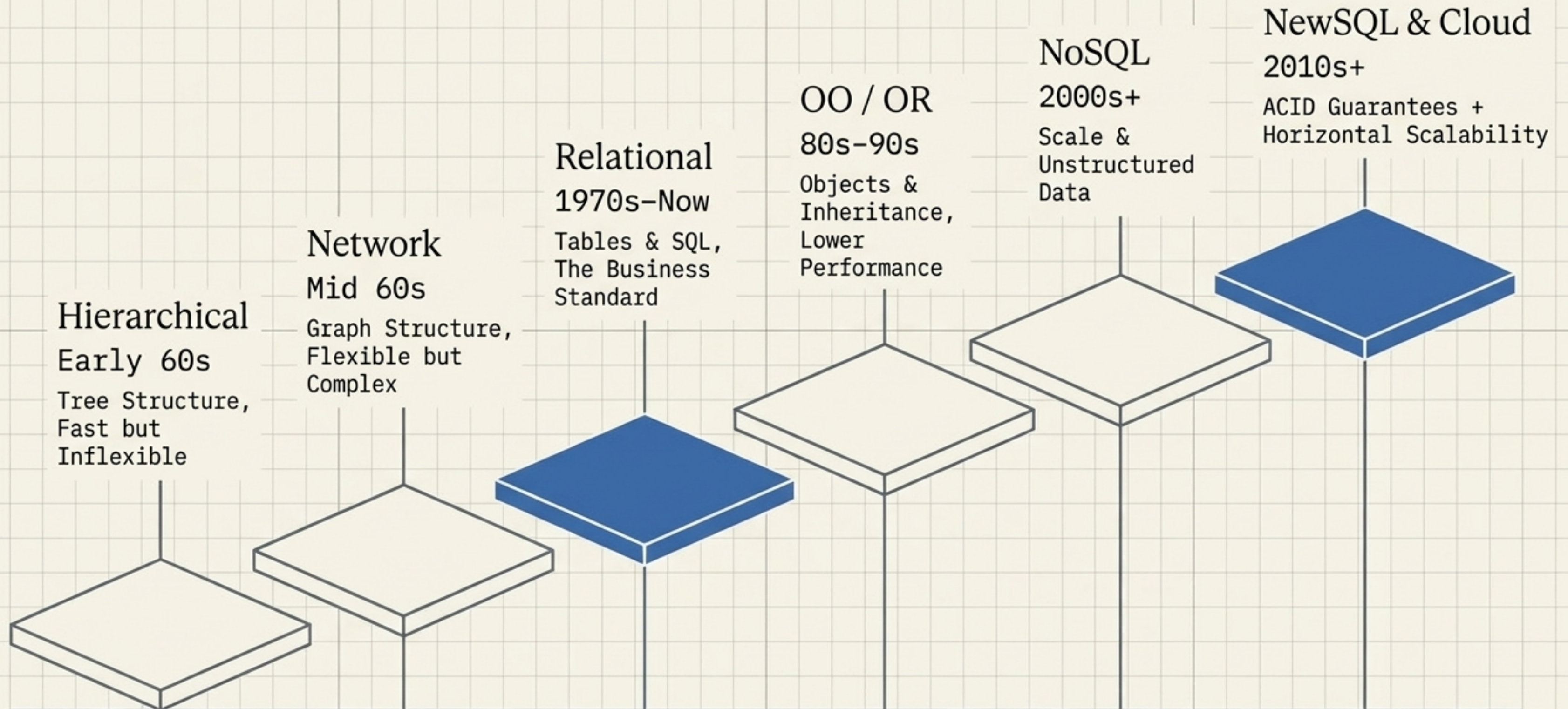


Poor Scaling

High volume shattered consistency and prevented concurrent institutional access.



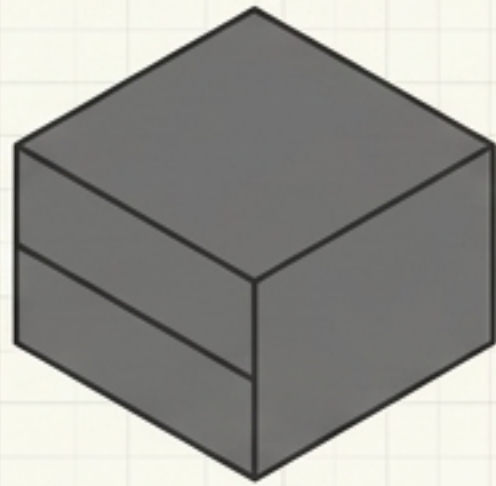
The Evolution of Database Systems



The Modern Database Era Matrix

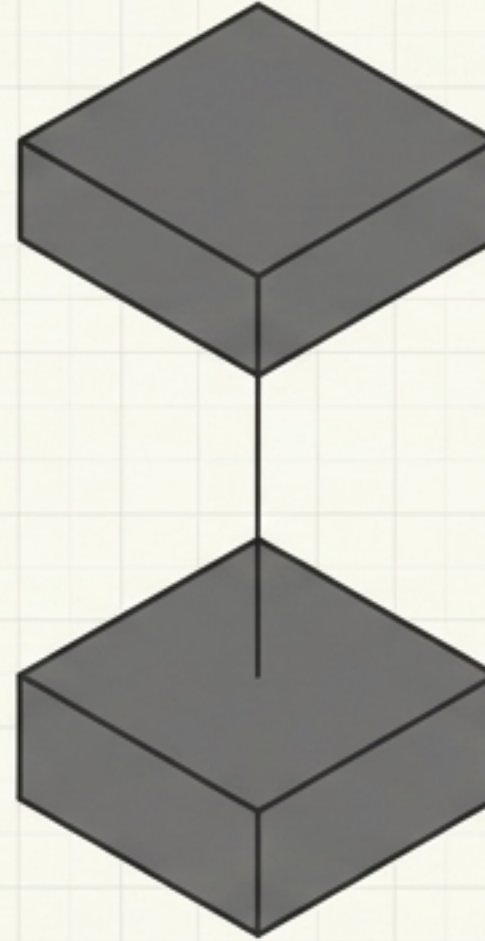
Relational (SQL)	Object-Oriented/Relational	NoSQL & Distributed
Core Structure: Tables, Rows, Columns Key Advantage: Simple, flexible retrieval (SQL). Target Use: Structured business data. (e.g., MySQL, Oracle, PostgreSQL).	Core Structure: Complex Data Types Key Advantage: Encapsulation & Inheritance. Target Use: Bridging OO programming with relational foundations.	Core Structure: Key-Value, Document, Column, Graph Key Advantage: High scalability and flexibility. Target Use: Large volumes of unstructured data. (e.g., Redis, MongoDB, Cassandra).

DBMS Architecture: Physical Foundations



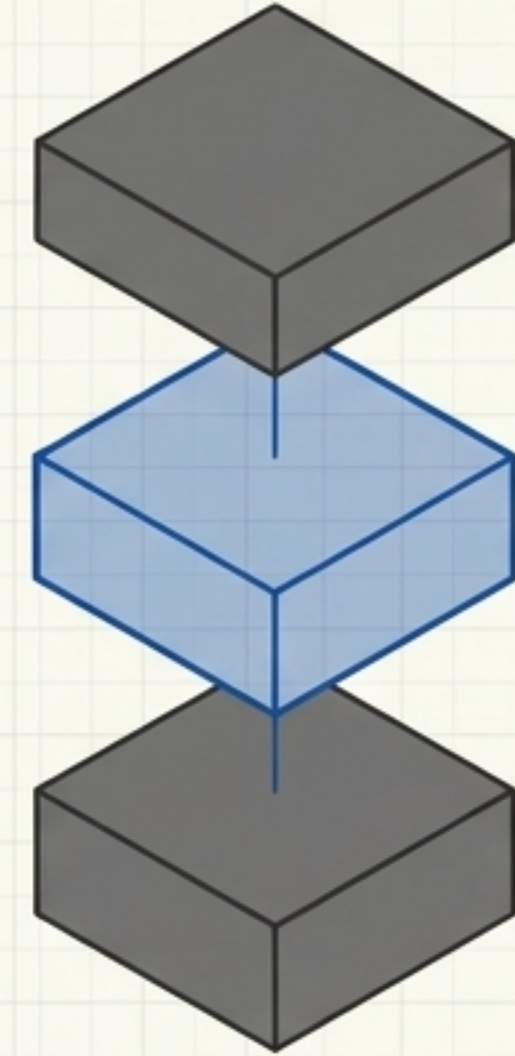
1-Tier

User + DBMS on the same machine. No network separation (e.g., local SQLite).



2-Tier

Client interacts with Server/DBMS via a network connection.
Classic client-server model.



3-Tier

Client talks to an App/Logic Layer, which talks to the Database.
Improves security, scalability, and maintainability (Modern Web Apps).

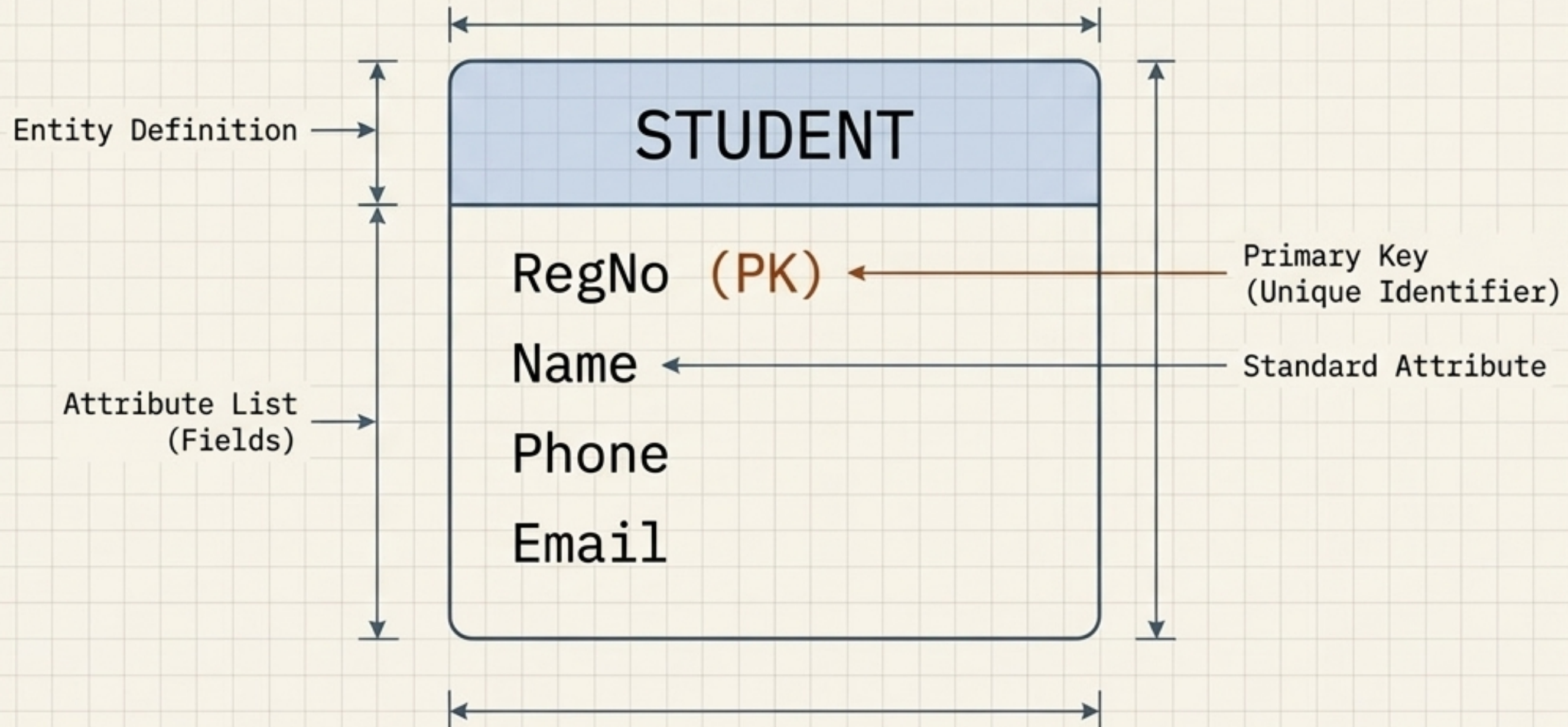
ER Modeling: The Top-Down Playbook

Database design requires identifying macro structures before micro details.

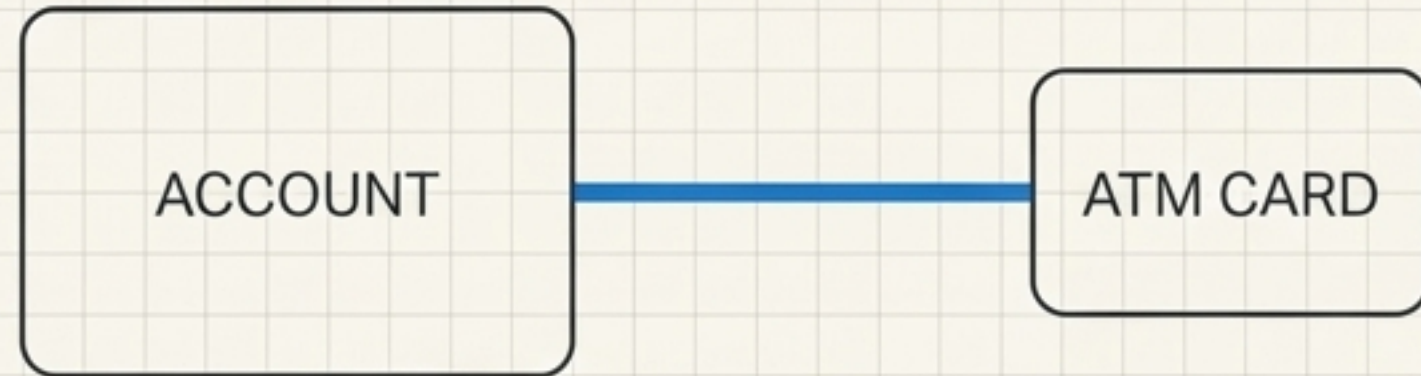


The Entity Blueprint

A set of objects sharing characteristics. An occurrence (instance) is one concrete object in that set. Names must be unique.



Entity Dependence: Strong vs. Weak



Strong Entity:

Can be uniquely identified by its own attributes (e.g., Employee, Account).

Stands completely alone.

Weak Entity:

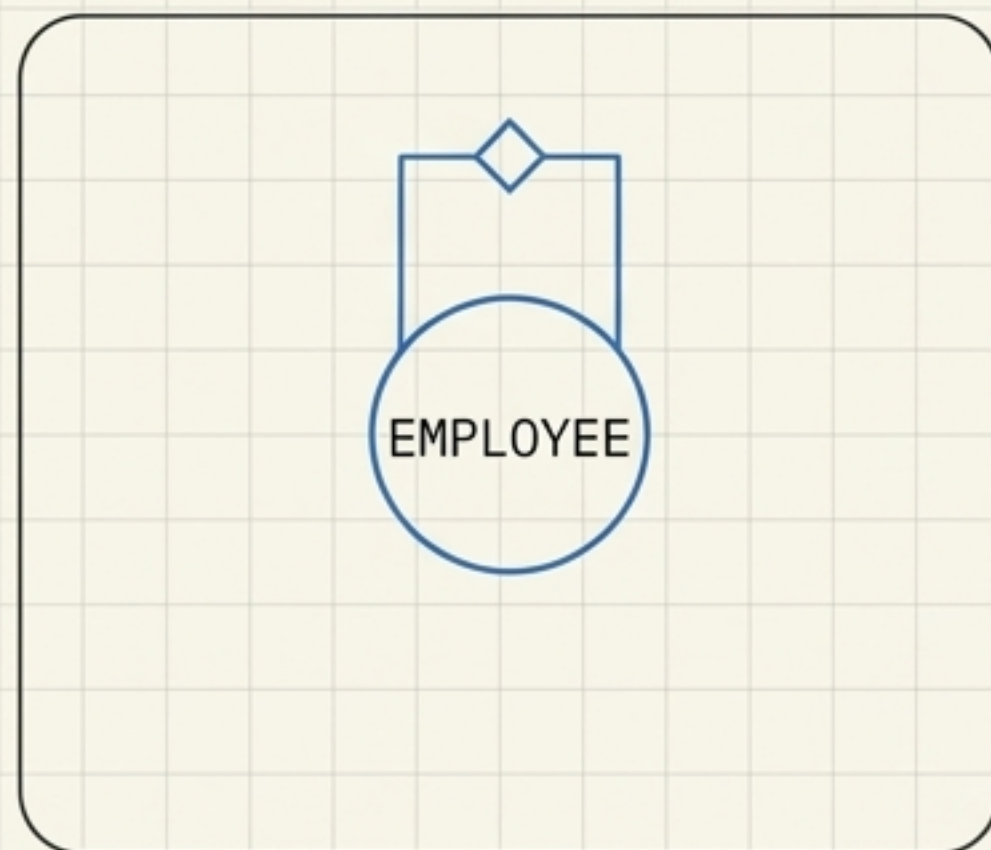
Depends entirely on a strong (owner) entity for identification.

The ATM Card Paradox:

An ATM card cannot be fully understood on its own. Without the owning Account context, the card number has no independent meaning in the bank's model.

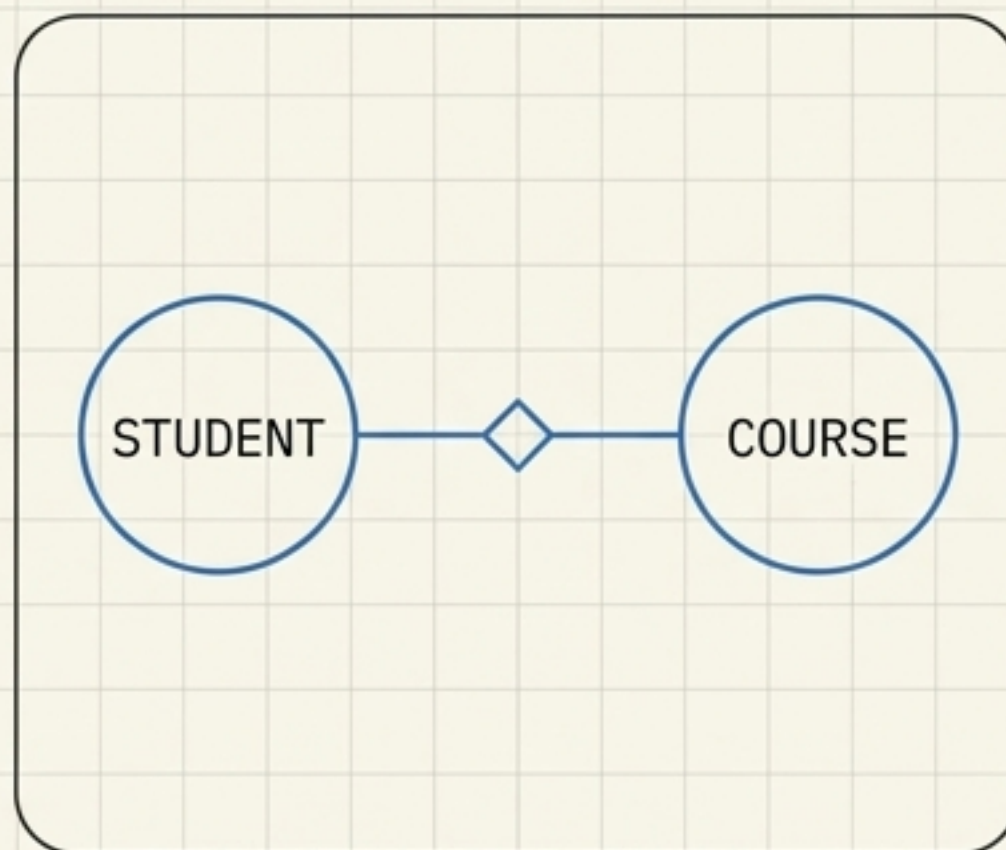
Relationship Node Graphs (Degrees)

The degree is the mathematical number of entity types participating in the relationship.



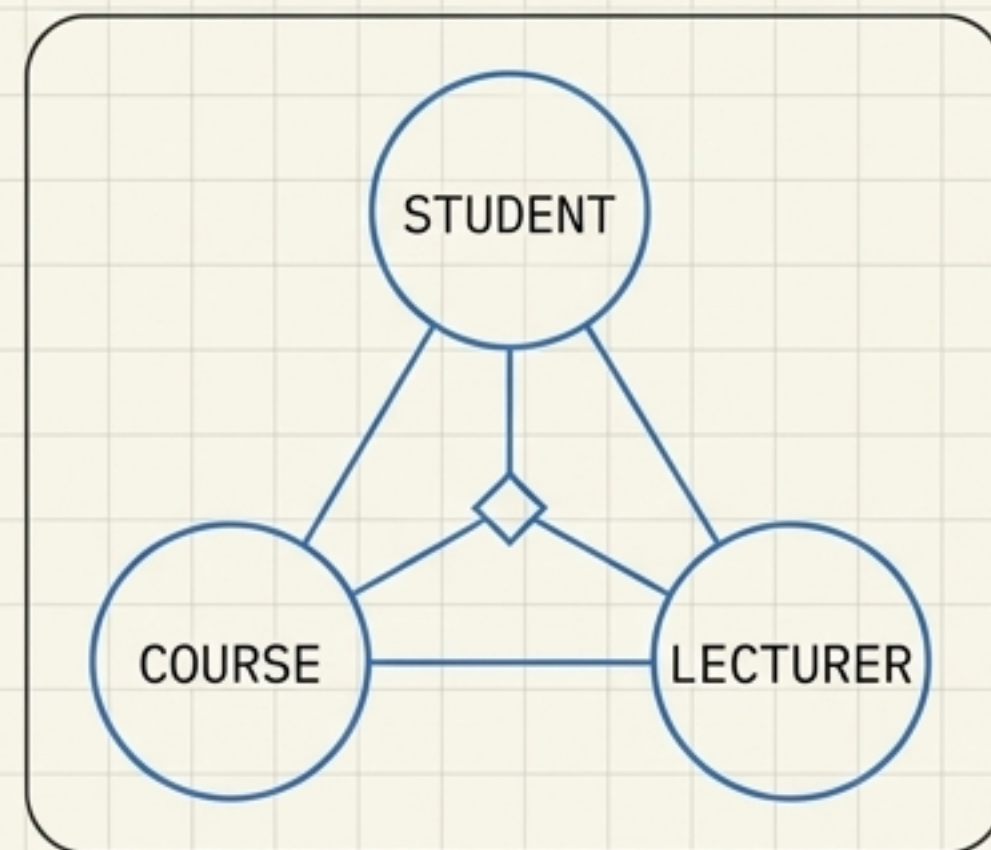
Unary (1 Entity)

Employee manages Employee.



Binary (2 Entities)

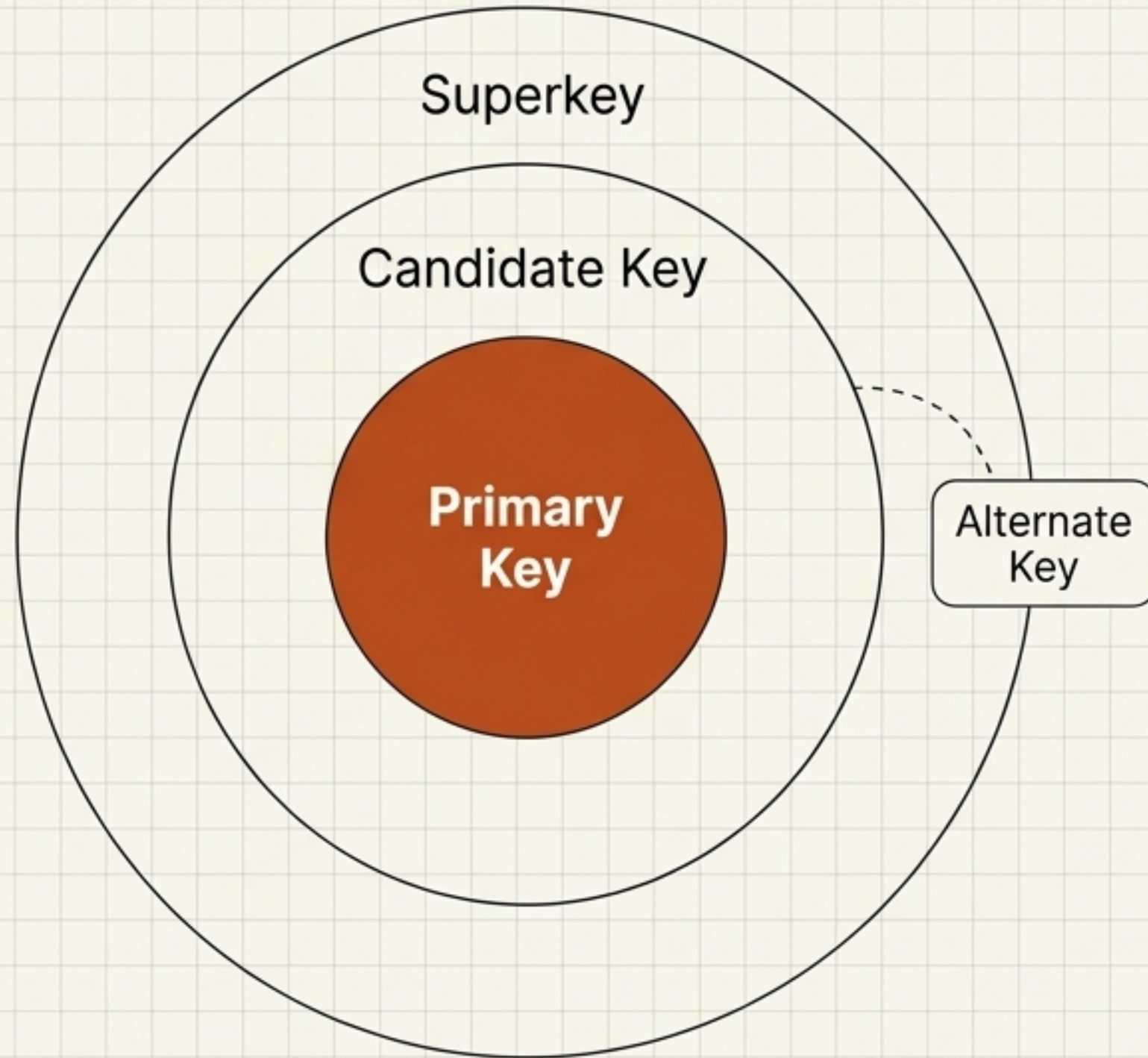
Student owns Course.



Ternary (3 Entities)

Student – Course – Lecturer.

The Taxonomy of Keys



Superkey

Set of attributes that uniquely identifies an entity (e.g., RegNo + ID + Phone).

Candidate Key

A minimal superkey. The pool of choices (e.g., RegNo, ID).

Primary Key (PK)

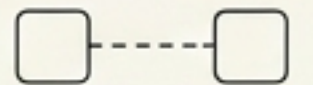
The exact candidate chosen to identify the entity (e.g., RegNo).

Alternate Key

The candidates left behind (not chosen as PK).

Foreign Key (FK)

Placed in one table to reference the PK of another, physically connecting the entities.



External Reference

Attribute Diagnostics

Single-valued vs. Multi-valued

Single holds one absolute value. **Multi** can hold multiple concurrent values.

Single: Gender

Multi: **Phone**: 01-..., 07-...

Simple vs. Composite

Simple cannot be subdivided. **Composite** breaks down into smaller, meaningful parts.

Simple: Price

Composite: **Full Name** → First + Last

Required vs. Optional

Required enforces **strict presence** constraints. **Optional** permits empty fields.

Required: **NOT NULL**

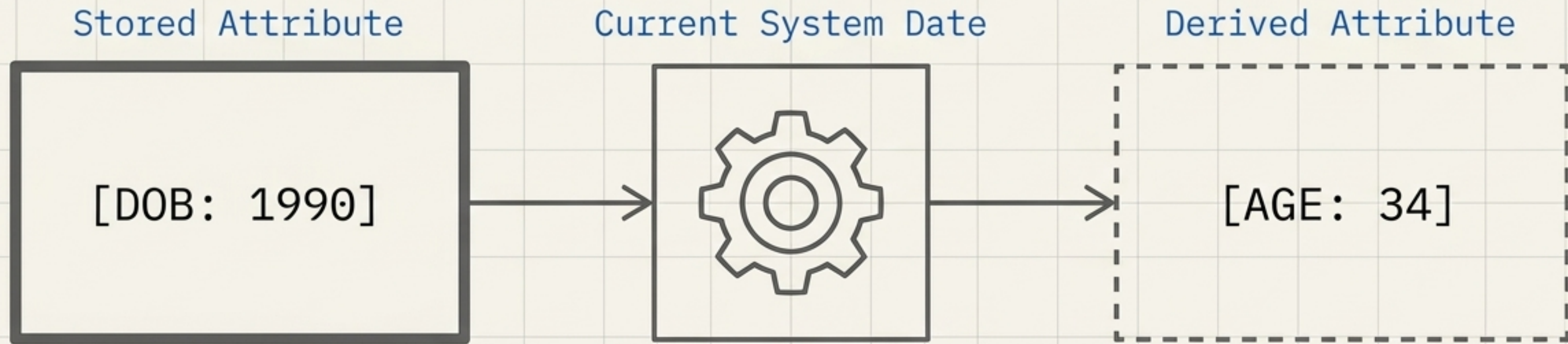
Optional: **NULL Allowed**

Domain

The complete set of all possible **legal values** permitted for a specific attribute.

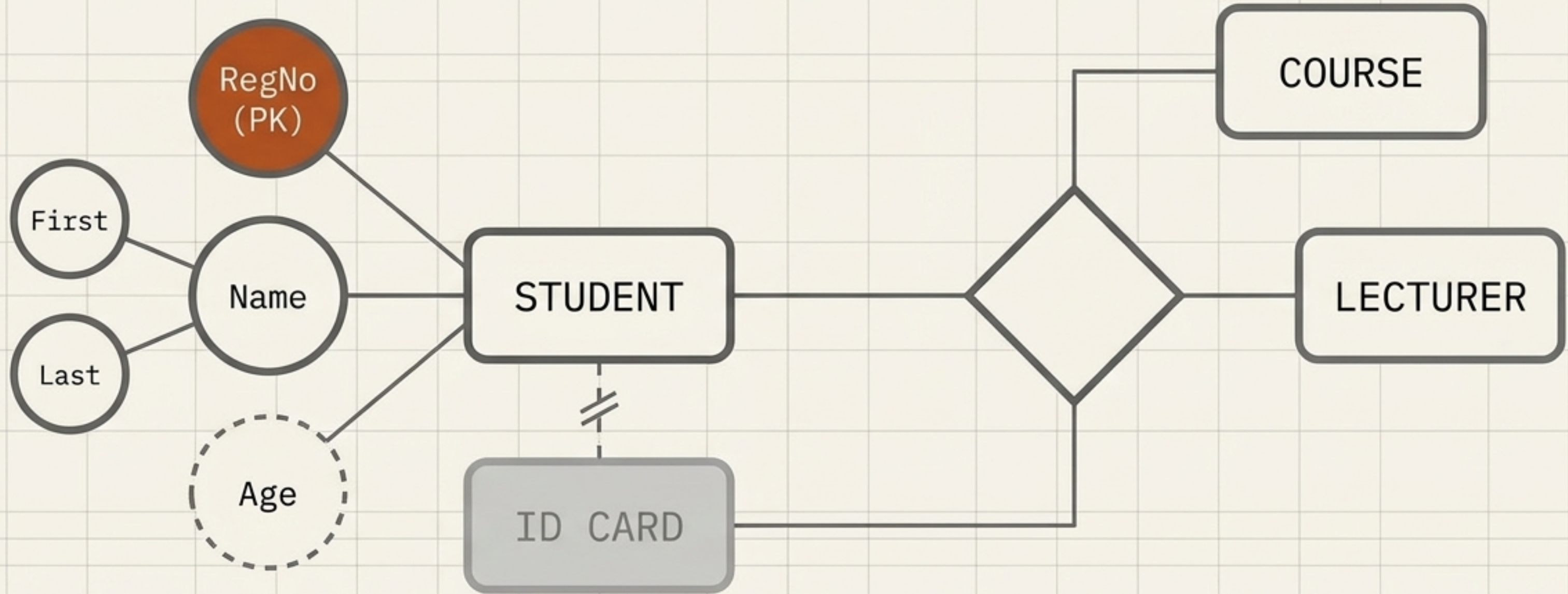
Domain of Gender = {**Male**, **Female**, **Other**}

The Derivation Engine



Key Insight: A derived attribute is calculated on the fly from other attributes. It is never stored permanently in the database schema.

Synthesis: The Complete ER Ecosystem



Every atomic concept—from constraints to architecture—serves this single blueprint, ensuring data remains consistent, scalable, and accessible.